

BACKEND TRAINING SERIES

Prisma + SQL Server + Node.js

การจัดการข้อมูลแบบครบวงจร
ด้วยตัวอย่างระบบ E-commerce

JavaScript

CommonJS

Express

Prisma ORM

SQL Server

สารบัญ (Agenda)

01	แนะนำ Prisma + SQL Server	02	Architecture & Business flow	03	ER Diagram ระบบ E-commerce
04	Setup โปรเจกต์ + Connection	05	schema.prisma & Models	06	Relation (1:1, 1:N)
07	db push vs migrate	08	Prisma Client + Studio + Seed	09	เชื่อมกับ Express
10	CRUD & Relation query	11	Filter, Sort, Pagination	12	Transaction, Validation, Best practice, Workshop

หัวใจของคอร์ส: ออกแบบ → เขียน schema → เชื่อม DB → ใช้งานจริงในระบบ E-commerce

Prisma คืออะไร?

1 Next-generation ORM สำหรับ Node.js

- ORM (Object-Relational Mapping) ที่เขียน query ด้วย JavaScript/TypeScript
- ใช้ไฟล์ `schema.prisma` เป็น single source of truth ของโครงสร้าง DB
- สร้าง type-safe Prisma Client ให้โดยอัตโนมัติ
- รองรับหลาย DB: SQL Server, PostgreSQL, MySQL, SQLite, MongoDB
- มีเครื่องมือครบ: Migrate, Studio, Client, Introspect
- ลดการเขียน SQL ซ้ำ ๆ และลด bug จาก column/ชนิดข้อมูลผิด

schema.prisma

ประกาศ datasource, generator และ model ทั้งหมดในไฟล์เดียว

Prisma Client

ไลบรารีที่ generate ขึ้นมา ใช้ query เช่น `findMany`, `create`, `update`

Prisma Migrate / Studio

สร้างตารางใน DB จริง และเปิด GUI ดูข้อมูลได้ทันที

Prisma เหมาะกับงานประเภทใด & ทำไมต้อง SQL Server

Prisma เหมาะกับ...

- REST / GraphQL API บน Node.js
- ระบบ E-commerce, CMS, SaaS, ERP ภายในองค์กร
- โปรเจกต์ที่ schema เปลี่ยนบ่อย ต้องการ migration ชัดเจน
- ทีมที่อยาก type-safe โดยไม่ต้องเขียน SQL เอง
- งานที่ต้องการ productivity สูง + อ่านโค้ดง่าย

ทำไมต้อง SQL Server?

- เป็น DB หลักในองค์กรไทยหลายแห่ง (SQL Server 2019/2022)
- รองรับ transaction, constraint, index ครบถ้วน
- ทำงานร่วมกับ Windows Auth / AD ได้ดี
- Prisma รองรับ Provider: sqlserver อย่างเป็นทางการ
- เหมาะกับงานระบบ enterprise ที่ต้องการความมั่นคง

ภาพรวม Architecture: Node.js + Prisma + SQL Server

Client / Browser / Mobile

ส่ง HTTP request → API

Express.js (Routes + Controllers)

รับ request, validate, เรียก service

Service Layer (Business Logic)

คำนวณราคา, จัดการ order, transaction

Prisma Client (ORM)

query.findMany(), \$transaction(), ฯลฯ

SQL Server Database

ตารางจริง + constraint + index

หลักการหลัก

- แยกชั้น routes / controller / service ชัดเจน
- Prisma Client อยู่ใน service layer เท่านั้น
- ไม่ query DB ตรงจาก controller
- Reusable PrismaClient 1 instance ทั้งแอป
- ใช้ .env แยก config dev / prod
- ทุก request ผ่าน validation ก่อนเข้าสู่ service

ภาพรวม Business Flow ของระบบ E-commerce



ข้อมูลหลัก 8 entity

User, Category, Product, Cart, CartItem, Order, OrderItem, Payment

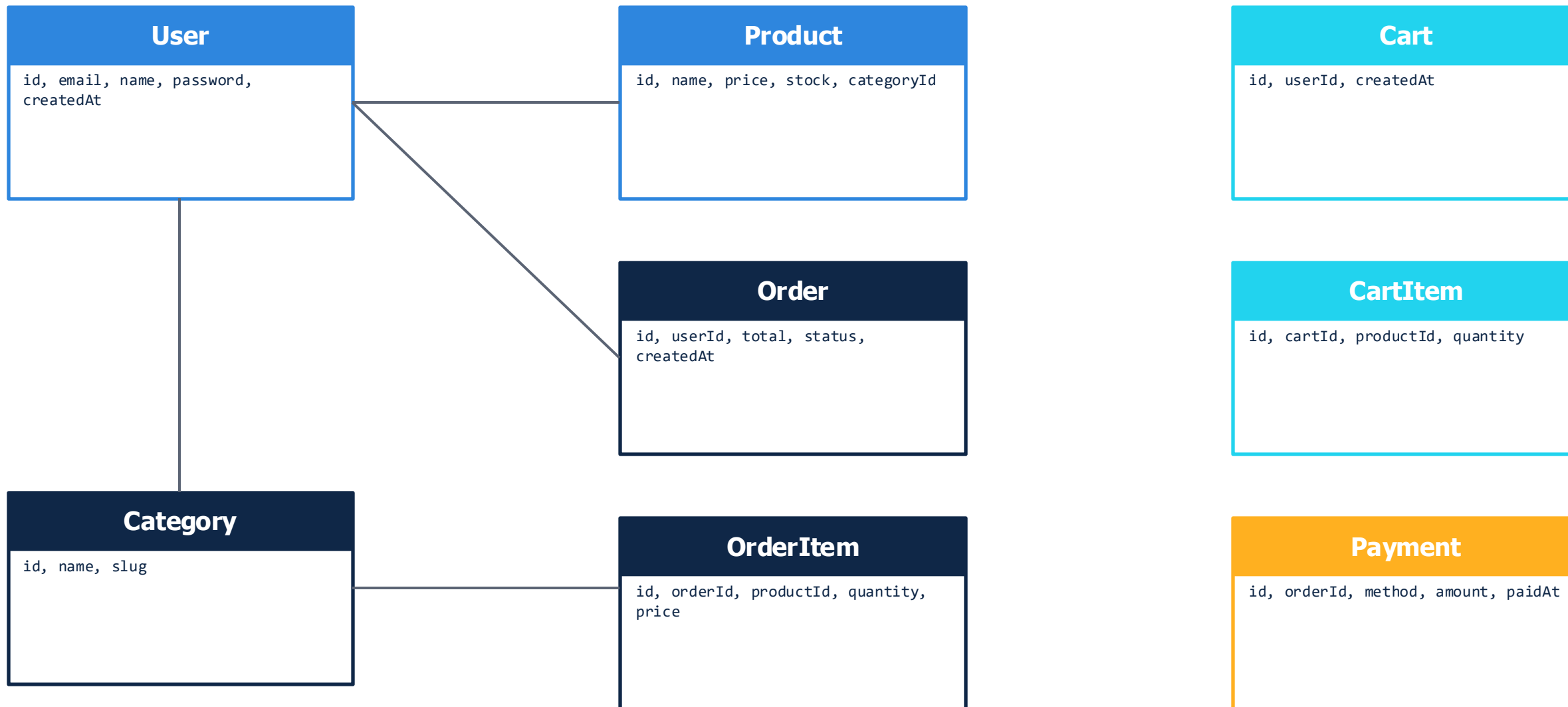
จุดสำคัญของระบบ

ต้องจัดการ stock และเงินอย่างถูกต้อง → ใช้ transaction บันทึก payment หลังสร้าง order สำเร็จเท่านั้น

สิ่งที่ Prisma ช่วย

สร้าง relation ครบ / query พร้อม include / \$transaction() รวมหลาย operation ไปด้วยกัน

ER Diagram ระบบ E-commerce



1 User → N Orders, 1 User → 1 Cart, 1 Category → N Products, 1 Order → N OrderItems, 1 Order → 1 Payment

ติดตั้งโปรเจกต์ Node.js + Prisma

- ต้องมี Node.js 18+ ติดตั้งในเครื่อง
- สร้างโฟลเดอร์โปรเจกต์ใหม่ + npm init
- ติดตั้ง Prisma CLI แบบ dev dependency
- ติดตั้ง @prisma/client สำหรับใช้ใน runtime
- รัน npx prisma init --datasource-provider sqlserver
- จะได้โฟลเดอร์ prisma/ และไฟล์ .env อัตโนมัติ
- Express เป็น framework หลักสำหรับสร้าง REST API

```
terminal - setup.sh

# 1) สร้างโปรเจกต์
mkdir ecommerce-api && cd ecommerce-api
npm init -y

# 2) ติดตั้ง dependency หลัก
npm install express dotenv
npm install prisma --save-dev
npm install @prisma/client

# 3) initialize Prisma
npx prisma init \
  --datasource-provider sqlserver

# โครงสร้างที่ได้
ecommerce-api/
├── prisma/
│   └── schema.prisma
├── .env
├── server.js
└── package.json
```

ตั้งค่า .env และ SQL Server Connection String

- Prisma อ่าน DATABASE_URL จากไฟล์ .env
- Format ของ SQL Server คือ
sqlserver://host:port;database=;user=;password=;
- ใส่ encrypt=true กรณีใช้ SSL (แนะนำใน production)
- ใส่ trustServerCertificate=true ถ้า self-signed (dev เท่านั้น)
- อย่า commit ไฟล์ .env ขึ้น git (เพิ่มใน .gitignore)
- แยกไฟล์ .env.development / .env.production ตาม environment

```
.env / .gitignore

# .env
DATABASE_URL="sqlserver://localhost:1433;\
database=ecommerce;\
user=sa;\
password=YourStrong!Passw0rd;\
encrypt=true;\
trustServerCertificate=true"

PORT=3000
NODE_ENV=development

# .gitignore
node_modules
.env
.env.*
!.env.example
```

schema.prisma: datasource & generator

- datasource กำหนดชนิดและที่อยู่ของฐานข้อมูล
- provider = "sqlserver" สำหรับ Microsoft SQL Server
- url อ่านค่าจาก env("DATABASE_URL")
- generator client บอกว่าจะสร้าง Prisma Client สำหรับ JavaScript
- provider = "prisma-client-js" เป็น default
- เมื่อแก้ schema → รัน npx prisma generate เพื่ออัปเดต client

```
prisma/schema.prisma

// prisma/schema.prisma

datasource db {
  provider = "sqlserver"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

// ทุกครั้งที่แก้ model ต้องรัน:
// npx prisma generate
// เพื่อ regenerate client
```

สร้าง Model และกำหนด Relation ใน schema.prisma

- แต่ละ model คือหนึ่งตารางใน DB
- ใส่ @id ให้ primary key, @unique ให้ column ที่ห้ามซ้ำ
- @default(autoincrement()) ใช้กับ Int id, @default(now()) กับ DateTime
- Decimal @db.Decimal(10,2) เก็บราคา (ห้ามใช้ Float เก็บเงิน)
- Relation ใช้คู่ field: scalar FK + navigation field
- 1:N → ใส่ many ใส่ foreign key, ใส่ one ใส่ array
- 1:1 → ใช้ @unique ที่ foreign key

```
prisma/schema.prisma

model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  name    String
  password String
  orders  Order[]
  cart    Cart?
  createdAt DateTime @default(now())
}

model Category {
  id      Int      @id @default(autoincrement())
  name    String
  slug    String   @unique
  products Product[]
}

model Product {
  id          Int      @id @default(autoincrement())
  name        String
  price       Decimal  @db.Decimal(10,2)
  stock       Int      @default(0)
  categoryId  Int
  category    Category @relation(fields: [categoryId],
                                references: [id])
  orderItems  OrderItem[]
  cartItems   CartItem[]
}
```

Model ต่อ: Cart, Order, OrderItem, Payment (1:1 vs 1:N)

แนวคิด Relation

- One-to-Many (1:N): User → Orders, Order → OrderItems
- One-to-One (1:1): User → Cart, Order → Payment (ใช้ @unique)
- FK field (เช่น userId) + navigation field (user)
- onDelete: Cascade ให้ลบลูกทั้งหมดเมื่อลบ parent
- พิจารณา soft delete: เพิ่ม field deletedAt DateTime?
- Index: @@index([field]) ช่วยให้ query เร็วขึ้น

```
prisma/schema.prisma (ต่อ)

model Cart {
  id      Int      @id @default(autoincrement())
  userId  Int      @unique // 1:1
  user    User     @relation(fields: [userId],
                             references: [id])

  items   CartItem[]
}

model Order {
  id      Int      @id @default(autoincrement())
  userId  Int
  total   Decimal  @db.Decimal(12,2)
  status  String   @default("PENDING")
  user    User     @relation(fields: [userId],
                             references: [id])

  items   OrderItem[]
  payment Payment?
  createdAt DateTime @default(now())
  @@index([userId, createdAt])
}

model Payment {
  id      Int      @id @default(autoincrement())
  orderId Int      @unique
  method  String
  amount  Decimal  @db.Decimal(12,2)
  paidAt  DateTime @default(now())
  order   Order    @relation(fields: [orderId],
                             references: [id])
}
```

Database Lifecycle: db push vs migrate

หัวข้อ	prisma db push	prisma migrate dev
เป้าหมาย	prototype / dev เร็ว ๆ	สร้าง migration history ถาวร
สร้างไฟล์?	ไม่สร้าง migration file	สร้างโฟลเดอร์ migrations/...
เหมาะกับ	ช่วง design schema ยังไม่นิ่ง	ทุกโปรเจกต์ที่ขึ้น production
ย้อนกลับ	ย้อนไม่ได้ (ทำ schema ตรง ๆ)	rollback ได้ ผ่าน migrate resolve
CI/CD	ไม่เหมาะ	เหมาะมาก: ใช้ migrate deploy

 terminal

```
# ช่วง dev - sync schema ไว ๆ
npx prisma db push
```

```
# ช่วง production-ready - สร้าง migration
npx prisma migrate dev --name init
```

Prisma Client · Studio · Seed ข้อมูลเริ่มต้น

npx prisma generate

สร้าง @prisma/client ใหม่ตาม schema
ต้องรันทุกครั้งหลังแก้ model

npx prisma studio

เปิด GUI ที่ localhost:5555
ดู/แก้ข้อมูลในทุกตารางได้ทันที

npx prisma db seed

รันไฟล์ prisma/seed.js
ใส่ข้อมูลเริ่มต้นให้ระบบ

```
prisma/seed.js

// prisma/seed.js
const { PrismaClient } = require('@prisma/client')
const prisma = new PrismaClient()

async function main () {
  await prisma.category.createMany({
    data: [
      { name: 'Electronics', slug: 'electronics' },
      { name: 'Books', slug: 'books' }
    ],
    skipDuplicates: true
  })

  await prisma.product.create({
    data: {
      name: 'Keyboard',
      price: 1299.99
    }
  })
}
```

เชื่อม Prisma Client กับ Express App

- สร้างไฟล์ prisma.js เพื่อ export PrismaClient 1 instance
- ป้องกันการ instance ใหม่ทุกครั้ง hot-reload ใน dev
- ใน server.js เรียกใช้ express + routes
- แยกโฟลเดอร์: routes / controllers / services ชัดเจน
- routes รู้จัก HTTP method, controller จัดการ req/res, service คุย DB

โครงสร้างโปรเจกต์

```

ecommerce-api/
├── prisma/
│   ├── schema.prisma
│   └── seed.js
├── src/
│   ├── prisma.js           ← shared PrismaClient
│   ├── routes/
│   │   └── productRoutes.js
│   ├── controllers/
│   │   └── productController.js
│   └── services/
│       └── productService.js
├── server.js
└── .env
  
```

```

src/prisma.js + server.js

// src/prisma.js (reusable client)
const { PrismaClient } = require('@prisma/client')
const prisma = global.__prisma || new PrismaClient({
  log: ['warn', 'error']
})
if (process.env.NODE_ENV !== 'production')
  global.__prisma = prisma
module.exports = prisma

// server.js
const express = require('express')
const productRoutes = require('./src/routes/productRoutes')

const app = express()
app.use(express.json())
app.use('/api/products', productRoutes)

const PORT = process.env.PORT || 3000
app.listen(PORT, () =>
  console.log('API ready on ' + PORT))
  
```

CRUD Product: Service Layer + Controller

- Service = คุยกับ Prisma, ไม่รู้เรื่อง req/res
- Controller = รับ req, validate, เรียก service, ตอบ res
- create / findMany / findUnique / update / delete
- ใช้ where, select, include ใดในทุก query
- return ข้อมูลเฉพาะที่จำเป็น เพื่อ performance

```

services/productService.js

// services/productService.js
const prisma = require('../prisma')

exports.list = () => prisma.product.findMany({
  include: { category: true }
})
exports.getById = (id) => prisma.product.findUnique({
  where: { id: Number(id) }
})
exports.create = (data) => prisma.product.create({ data })
exports.updateStock = (id, stock) =>
  prisma.product.update({
    where: { id: Number(id) },
    data: { stock }
  })
exports.remove = (id) =>
  prisma.product.delete({ where: { id: Number(id) } })

```

```

controllers + routes

// controllers/productController.js
const service = require('../services/productService')

exports.list = async (req, res) => {
  const items = await service.list()
  res.json(items)
}

exports.create = async (req, res) => {
  try {
    const p = await service.create(req.body)
    res.status(201).json(p)
  } catch (err) {
    res.status(400).json({ message: err.message })
  }
}

// routes/productRoutes.js
const router = require('express').Router()
const c = require('../controllers/productController')
router.get('/', c.list)
router.post('/', c.create)
router.patch('/:id/stock', c.updateStock)
router.delete('/:id', c.remove)
module.exports = router

```

Relation Query: Order พร้อม OrderItems และ User

- include ดึง relation มาด้วย (JOIN)
- select เลือกเฉพาะ column ที่ต้องการ
- nested include ซ้อนได้หลายชั้น เช่น items.product.category
- หลีกเลี่ยงการ include ทุกอย่าง → ดึงเฉพาะที่ใช้จริง (ประหยัด bandwidth)
- ใช้ orderBy ได้ในทั้ง parent และ relation

JSON response (ย่อ)

```
{
  "id": 101, "total": 2580.00, "status": "PAID",
  "user": { "id": 3, "email": "a@x.com" },
  "items": [
    { "quantity": 2, "product": { "name": "Keyboard" } },
    { "quantity": 1, "product": { "name": "Mouse" } }
  ],
  "payment": { "method": "CREDIT_CARD", "amount": 2580 }
}
```

```
services/orderService.js

// services/orderService.js
const prisma = require('../prisma')

exports.getOrderDetail = (id) =>
  prisma.order.findUnique({
    where: { id: Number(id) },
    include: {
      user: { select: { id: true, email: true, name: true } },
      items: {
        include: { product: { select: { name: true, price: true } } },
        orderBy: { id: 'asc' }
      },
      payment: true
    }
  })

exports.listOrdersByUser = (userId) =>
  prisma.order.findMany({
    where: { userId: Number(userId) },
    orderBy: { createdAt: 'desc' },
    include: { items: true, payment: true }
  })
```

Filtering, Sorting, Pagination, Search

- where: เงื่อนไขกรอง เช่น equals, gt, lt, contains, in
- orderBy: จัดเรียงได้หลาย field เช่น [{price:'asc'}, {id:'desc'}]
- skip + take: pagination มาตรฐาน (offset + limit)
- cursor: pagination แบบ cursor-based สำหรับ data จำนวนมาก
- contains + mode: 'insensitive' ใช้ค้นหาแบบไม่สนตัวพิมพ์
- combine AND / OR ได้ เช่น { AND: [...], OR: [...] }

```
services/productService.js

// services/productService.js
exports.search = async ({ q, categoryId, minPrice,
                        maxPrice, sort='desc', page=1, size=10 }) => {

  const where = {
    AND: [
      q ? { name: { contains: q } } : {},
      categoryId ? { categoryId: Number(categoryId) } : {},
      minPrice ? { price: { gte: Number(minPrice) } } : {},
      maxPrice ? { price: { lte: Number(maxPrice) } } : {},
      { deletedAt: null } // soft-delete filter
    ]
  }

  const [total, items] = await Promise.all([
    prisma.product.count({ where }),
    prisma.product.findMany({
      where,
      orderBy: { price: sort },
      skip: (page - 1) * size,
      take: Number(size),
      select: { id: true, name: true, price: true, stock: true }
    })
  ])

  return { total, page, size,
    pages: Math.ceil(total / size), items }
}
```

Transaction: ขั้นตอนการสั่งซื้อที่ปลอดภัย

- `prisma.$transaction([...])` รวมหลาย operation เป็นหน่วยเดียว
- หรือใช้แบบ interactive: `$transaction(async (tx) => {...})`
- ถ้าขั้นตอนใด throw → rollback ทั้งหมดอัตโนมัติ
- เหมาะสำหรับ: สร้าง Order + ลด stock + บันทึก Payment
- ตรวจสอบ stock ก่อน; ถ้าไม่พอให้ throw → ไม่หัก stock, ไม่สร้าง payment

ข้อควรระวัง

อย่าตรวจสอบ stock นอก transaction แล้วค่อยหักใน transaction — อาจเกิด race condition ตรวจสอบ + หัก ภายใน block เดียวกัน และพิจารณา isolation level: Serializable สำหรับระบบที่ต้องแม่นยำสูง

```
services/orderService.js - $transaction

// services/orderService.js
exports.createOrder = async (userId, items) => {
  return prisma.$transaction(async (tx) => {
    // 1) ตรวจสอบ + หัก stock ที่ละ item
    let total = 0
    for (const it of items) {
      const p = await tx.product.findUnique({
        where: { id: it.productId }
      })
      if (!p || p.stock < it.quantity)
        throw new Error('Out of stock: ' + it.productId)
      total += Number(p.price) * it.quantity
      await tx.product.update({
        where: { id: it.productId },
        data: { stock: { decrement: it.quantity } }
      })
    }

    // 2) สร้าง order + items
    const order = await tx.order.create({
      data: {
        userId, total, status: 'PENDING',
        items: { create: items.map(i => ({
          productId: i.productId,
          quantity: i.quantity,
          price: i.price
        })))
      }
    },
    include: { items: true }
  })

  // 3) บันทึก payment
  await tx.payment.create({
    data: { orderId: order.id, method: 'CREDIT_CARD',
      amount: total }
  })

  return order
}
```

Validation และ Error Handling

- Validate ก่อนเข้าสู่ service เสมอ (Zod / Joi / express-validator)
- ตรวจสอบ, required, ความยาว, format (email, uuid)
- ใช้ try/catch รอบ service call แล้ว map error → HTTP status
- Prisma throw PrismaClientKnownRequestError พร้อม code (P2002, P2025)
- P2002 = unique violation, P2025 = not found
- สร้าง central error middleware สำหรับรวม error ทั้งหมด

```
error + validation

// middlewares/errorHandler.js
const { Prisma } = require('@prisma/client')

module.exports = (err, req, res, next) => {
  console.error(err)
  if (err instanceof Prisma.PrismaClientKnownRequestError) {
    if (err.code === 'P2002')
      return res.status(409).json({ message: 'Duplicate field' })
    if (err.code === 'P2025')
      return res.status(404).json({ message: 'Not found' })
  }
  res.status(500).json({ message: 'Internal server error' })
}

// controllers/productController.js
const { z } = require('zod')
const schema = z.object({
  name: z.string().min(1),
  price: z.number().positive(),
  stock: z.number().int().min(0),
  categoryId: z.number().int()
})

exports.create = async (req, res, next) => {
  try {
    const data = schema.parse(req.body)
    const p = await service.create(data)
    res.status(201).json(p)
  } catch (err) { next(err) }
}
```

Best Practices & Common Mistakes สำหรับ Production

Do — Best Practices

- ใช้ PrismaClient single instance ทั้งแอป
- แยก routes / controllers / services / prisma.js
- Migrate ทุกครั้ง (อย่าใช้ db push บน prod)
- ใช้ Decimal เก็บเงิน อย่าใช้ Float
- ใช้ transaction กับงานที่ต้องปรับหลายตาราง
- Validate input ก่อนเข้าสู่ Prisma
- เพิ่ม @@index สำหรับ field ที่ query บ่อย
- Soft delete ด้วย deletedAt แทน hard delete
- เปิด log: ['warn','error'] + APM monitoring
- ตั้ง connection_limit ใน DATABASE_URL

Don't — Common Mistakes

- new PrismaClient() ทุก request → connection leak
- query DB ตรงจาก controller → ทดสอบยาก
- ลืมรัน prisma generate หลังแก้ schema
- ใช้ include ลึกเกินไป → query ช้า
- ไม่ใช่ transaction ตอนหัก stock + สร้าง order
- เก็บ password แบบ plain text (ต้อง hash ด้วย bcrypt)
- commit ไฟล์ .env ขึ้น git
- ไม่จัดการ error → ตอบ 500 ทุกกรณี
- ใช้ Float เก็บเงิน → ราคาผิดเพราะ rounding
- ใช้ db push บน production → รัน migration ล้ม

SUMMARY & NEXT STEPS

สรุปและก้าวต่อไป

01

เข้าใจ Prisma + SQL Server

schema.prisma, Provider sqlserver, Prisma Client

02

ออกแบบ DB ได้

Model, Relation 1:1 / 1:N, @@index, Decimal

03

สร้าง API จริงได้

Express + routes/controllers/services + CRUD + pagination

04

คุม data ถูกต้อง

\$transaction, validation, error handling, best practices

Next Steps:

- ① ทำ Workshop — สร้าง E-commerce API ของตัวเอง ② เพิ่ม auth ด้วย JWT ③ ทำ Docker + CI/CD ④ เพิ่ม unit/integration test (Jest + supertest)

ขอบคุณที่ติดตามจนจบ — Happy coding! 🚀